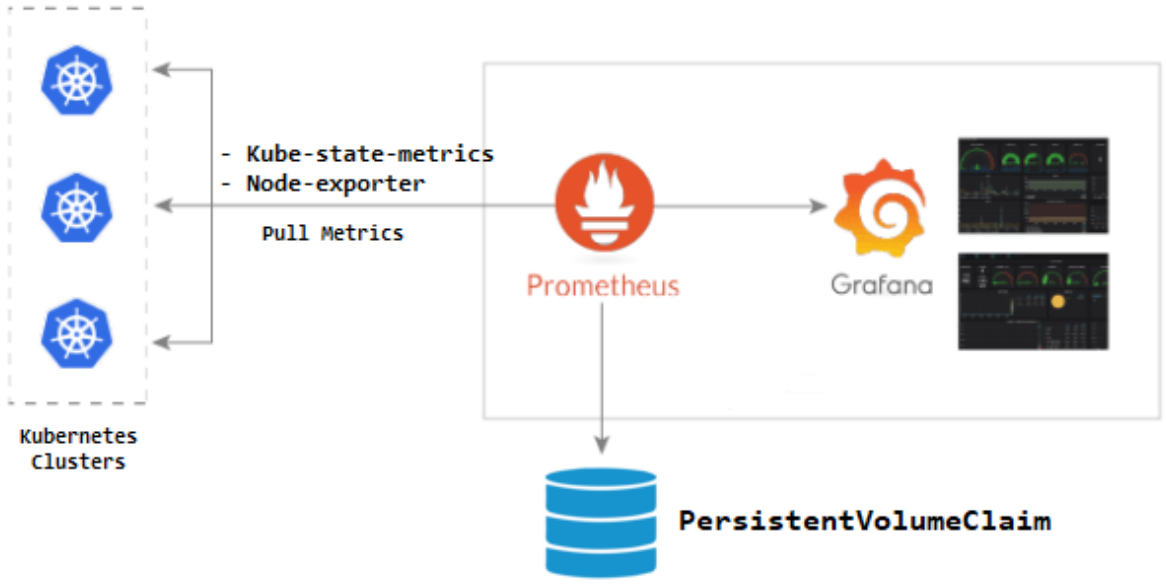
Setup Grafana On Kubernetes Cluster

Grafana is a free and open source visualisation tool that provides various dashboards, charts, graphs, alerts for the particular data source. Grafana allows us to query, visualise, explore metrics and set alerts for the data sources. We can also create our own dynamic dashboard for visualisation and monitoring. We can save the dashboards and share it with other members also. We can also import external dashboards.

architecture of Grafana and prometheus :



Tasks:

1. Setup Grafana (kubernetes)

2. Grafana Service

3. Connecting To Grafana Dashboard

4. Exposing Grafana Using Ingress

5. Create Grafana Dashboards

STEP 1 :

Setup Grafana (kubernetes)

Node : master-node

create a directory :

mkdir grafana-kube

cd grafana-kube

Create a grafana-datasource-config :

Create a file named grafana-datasource-config.yaml :

sudo nano grafana-datasource-config.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: grafana-datasources
  namespace: monitoring
data:
  prometheus.yaml: |-
    {
       "apiVersion": 1,
       "datasources": [
          {
             "access":"proxy",
              "editable": true,
              "name": "prometheus",
              "orgId": 1,
              "type": "prometheus",
              "url": "http://prometheus-service.monitoring.svc:8080",
              "version": 1
          }
       ]
    }
```

kubectl apply -f grafana-datasource-config.yaml

Create a file named grafana-deployment.yaml:

sudo nano grafana-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: grafana
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: grafana
  template:
    metadata:
      name: grafana
      labels:
        app: grafana
    spec:
      containers:
      - name: grafana
        image: grafana/grafana:latest
        ports:
        - name: grafana
          containerPort: 3000
        resources:
          limits:
            memory: "1Gi"
            cpu: "1000m"
          requests:
            memory: 500M
            cpu: "500m"
        volumeMounts:
          - mountPath: /var/lib/grafana
            name: grafana-storage
          - mountPath: /etc/grafana/provisioning/datasources
            name: grafana-datasources
            readOnly: false
      volumes:
        - name: grafana-storage
          emptyDir: {}
        - name: grafana-datasources
          configMap:
              defaultMode: 420
              name: grafana-datasources
```

kubectl apply -f grafana-deployment.yaml

check the created deployment :

kubectl get pods -n monitoring

STEP 2 :

Grafana Service

Create a file named grafana-service.yaml:

sudo nano grafana-service.yaml

Add this configuration to the file:

```
apiVersion: v1
kind: Service
metadata:
  name: grafana
  namespace: monitoring
  annotations:
     prometheus.io/scrape: 'true'
     prometheus.io/port:   '3000'
spec:
  selector:
    app: grafana
  type: NodePort
  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 32100
```

kubectl apply -f grafana-service.yaml

check the created service :

kubectl get service --namespace=monitoring

STEP 3 :

Connecting To Grafana Dashboard

Exposing Grafana as a Service NodePort

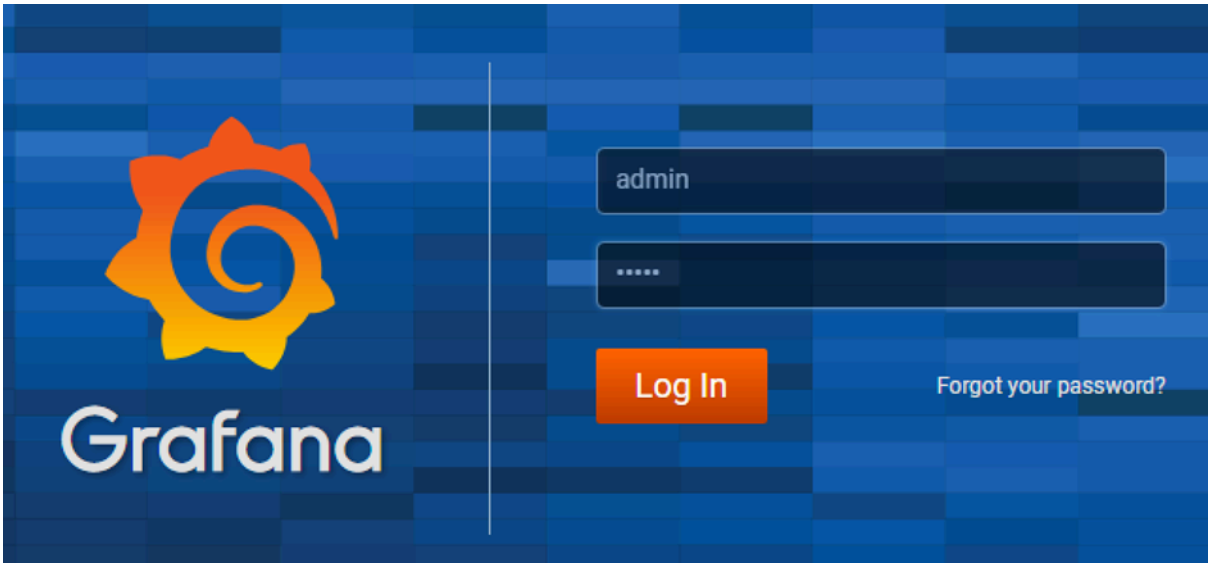
expose Grafana on all kubernetes node IP's on port 32100

you can access the Grafana dashboard using any of the Kubernetes nodes IP on port 32100

example :

http://IP-master-node:32100

http://x.x.x.x:32100



Default logins are:

Username: admin

Password: admin

STEP 4 :

Exposing Grafana Using Ingress

Check ingress controller :

kubectl get pods -n ingress-nginx

The container (nginx-controller) must be in the Runnig position

Create a file named grafana-ingress.yaml :

sudo nano grafana-ingress.yaml

Add this configuration to the file :

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: grafana-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: grafana-test.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: grafana
                port:
                  number: 3000
```

kubectl apply -f grafana-ingress.yaml --namespace=monitoring

Check ingress :

kubectl get ingress -n monitoring

STEP 5 :

Create Grafana Dashboards

Creating specific dashboards to each field in Kubernetes :

1_ Deployments

2_ HPA

3_ Nodes

4_ Pods

5_ Services

Dashboards new dashboard

Name Dashboard : Deployment dashboard

Name Panel 1 : kube_deployment_created
Mode : Time series

Set metrics :

kube_deployment_created

options Legend = custome

Add label : {{deployment}}

Name Panel 2: kube_deployment_spec_paused
Mode : Time series

Set metrics :

kube_deployment_spec_paused

options Legend = custome

Add label : {{deployment}}

Name Panel 3: kube_deployment_status_replicas
Mode : Gauge

Set metrics :

kube_deployment_status_replicas

options Legend = custome

Add label : {{deployment}}

Name Panel 4: kube_deployment_status_replicas_ready

Mode : Gauge

Set metrics :

kube_deployment_status_replicas_ready

options Legend = custome

Add label : {{deployment}}

Name Panel 5: kube_deployment_status_replicas_available

Mode : Gauge

Set metrics :

kube_deployment_status_replicas_available

options Legend = custome

Add label : {{deployment}}

Name Panel 6: kube_deployment_status_replicas_unavailable

Mode : Gauge

Set metrics :

kube_deployment_status_replicas_unavailable

options Legend = custome

Add label : {{deployment}}

Name Dashboard : **HPA dashboard**

Name Panel 1 : kube_horizontalpodautoscaler_info

Mode : Gauge

Set metrics :

kube_deployment_created

options Legend = custome

Add label : {{deployment}}

Name Panel 2 : kube_horizontalpodautoscaler_status_current_replicas

Mode : Gauge

Set metrics :

kube_horizontalpodautoscaler_status_current_replicas

options Legend = custome

Add label : {{deployment}}

Name Panel 3 : kube_horizontalpodautoscaler_spec_min_replicas

Mode : stat

Set metrics :

kube_horizontalpodautoscaler_spec_min_replicas

options Legend = custome

Add label : {{horizntalpodautoscaler}}

Name Panel 4 : kube_horizontalpodautoscaler_spec_max_replicas

Mode : stat

Set metrics :

kube_horizontalpodautoscaler_spec_max_replicas

options Legend = custome

Add label : {{horizntalpodautoscaler}}

Name Panel 5 : kube_horizontalpodautoscaler_spec_target_metric

Mode : stat

Set metrics :

kube_horizontalpodautoscaler_spec_target_metric

options Legend = custome

Add label : {{horizntalpodautoscaler}}

Name Panel 6 : kube_horizontalpodautoscaler_status_desired_replicas

Mode : stat

Set metrics :

kube_horizontalpodautoscaler_status_desired_replicas

options Legend = custome

Add label : {{horizntalpodautoscaler}}

Name Dashboard : **Node dashboard**

Name Panel 1 : kube_node_info

Mode : Bar chart

Set metrics :

kube_node_info

options Legend = custome

Add label : {{node}}

Name Panel 2 : kube_node_created

Mode : time serices

Set metrics :

kube_node_created

options Legend = custome

Add label : {{node}}

Name Panel 3 : kube_node_spec_unschedulable

Mode : Gauge

Set metrics :

kube_node_spec_unschedulable

options Legend = custome

Add label : {{node}}

add a new panel

Name Panel 4 : kube_node_spec_taint

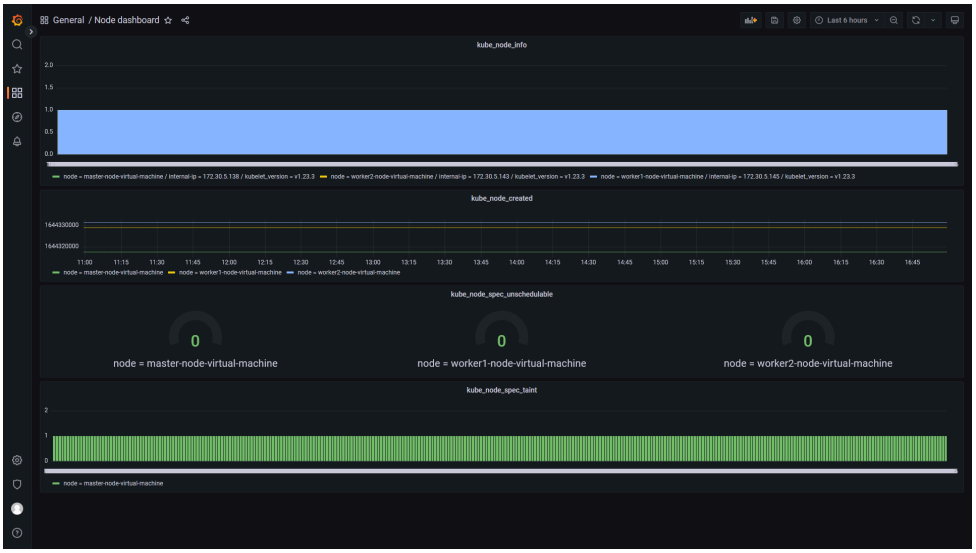
Mode : Bar chart

Set metrics :

kube_node_spec_taint

options Legend = custome

Add label : {{node}}



Name Dashboard : **Pod dashboard**

Name Panel 1 : kube_pod_container_info

Mode : pie chart

Set metrics :

kube_pod_container_info

options Legend = custome

Add label : {{container}}

Name Panel 2 : kube_pod_container_status_running

Mode : gauge

Set metrics :

kube_pod_container_status_running

options Legend = custome

Add label : {{container}}

Name Panel 3 : kube_pod_container_status_terminated

Mode : gauge

Set metrics :

kube_pod_container_status_terminated

options Legend = custome

Add label : {{container}}

Name Panel 4 : kube_pod_info

Mode : pie chart

Set metrics :

kube_pod_info

options Legend = custome

Add label : {{pod}}

Name Panel 5 : kube_pod_init_container_status_running

Mode : gauge

Set metrics :

kube_pod_init_container_status_running

options Legend = custome

Add label : {{container}}/{{pod}}

Name Dashboard : service dashboard

Name Panel 1 : kube_service_created

Mode : pie chart

Set metrics :

kube_service_created

options Legend = custome

Add label : {{service}}/{{namespace}}

Name Panel 2 : kube_service_info

Mode : time series

Set metrics :

kube_service_info

options Legend = custome

Add label : {{service}}/{{cluster_ip}}

Name Panel 3 : kube_service_spec_type

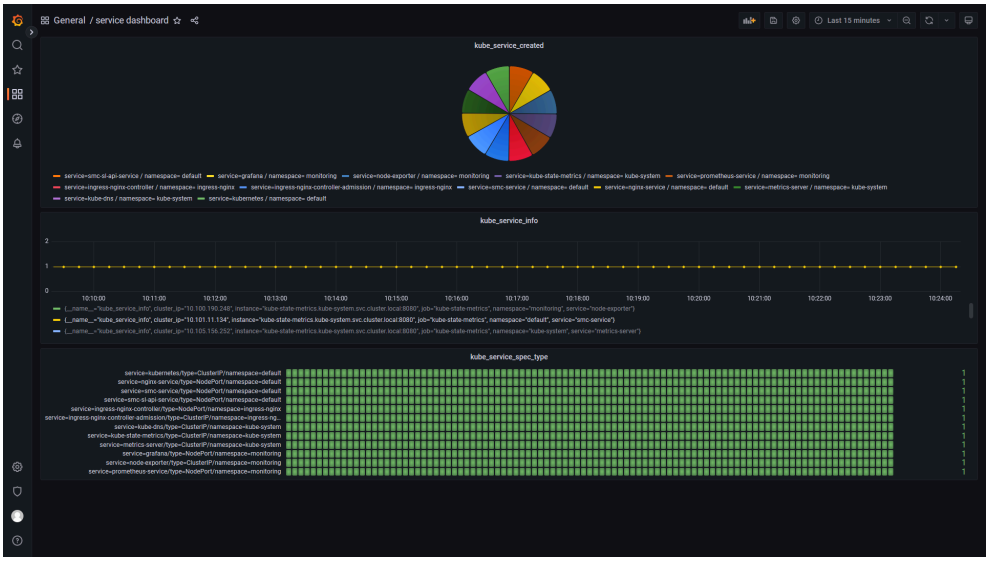
Mode : bar gauge

Set metrics :

kube_service_spec_type

options Legend = custome

Add label : {{service}}/{{type}}



Successful

END